МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра математического обеспечения ЭВМ

Цифровая обработка изображений

Лабораторная работа № 2

Студент: Дутов Р. В.

Преподаватель: Сырых А. С.

Воронеж 2026

Содержание

1. Постановка задачи..... 3
2. Математическое описание алгоритма..... 4
2.1. Алгоритмы выделения контуров..... 4
2.2. Линейные усредняющие фильтры..... 8
3. Программное описание алгоритма..... 10
4. Результат работы программы..... 11
5. Заключение..... 15
Приложение А..... 16
Приложение Б..... 19

1. Постановка задачи

1) Вариант № 1. Найти границы объектов тремя способами:
    a) с помощью лапласиана;
    b) с помощью разностей Робертса;
    c) с помощью градиента Собела.
2) Вариант № 2. Выполнить пространственное сглаживание изображения с гауссовым шумом. Сравнить равномерное усреднение и взвешенный фильтр Гаусса; шум сформировать программно.

2. Математическое описание алгоритма

Растровое изображение задаётся таблицей яркостей I(x, y). Координаты принимают целые значения: 0 ≤ x < W и 0 ≤ y < H, где W — ширина, H — высота. Для восьмибитного изображения яркость лежит в пределах 0–255. Если вход содержит цветовые каналы, сначала вычисляется полутоновая яркость по соотношению (1):

$$Y(x,y)=0.299*R(x,y)+0.587*G(x,y)+0.114*B(x,y) \qquad (1)$$

2.1.Алгоритмы выделения контуров

Граница объекта соответствует быстрому изменению яркости соседних пикселей. Для обнаружения такого изменения можно использовать приближённые первые либо вторые производные.

Оператор Собела

Метод Собела оценивает две составляющие градиента. Дифференцирование по одной оси сочетается со сглаживанием по другой, что частично уменьшает влияние случайных колебаний. Для расчёта служат два ядра 3×3, приведённые в (2):

$$G_x=\begin{pmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{pmatrix}, G_y=\begin{pmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{pmatrix} \qquad (2)$$

Длину полученного вектора определяем по выражению (3). Перед записью в восьмибитное изображение применяем ограничение (4):

$$G(x,y)=\sqrt{G_x^2(x,y)+G_y^2(x,y)} \qquad (3)$$

$$I_{out}(x,y)=min\left(255, max\left(0,\left[G(x,y)\right]\right)\right) \qquad (4)$$

Алгоритм 1. Оператор Собела

1) Вычислить яркость каждого пикселя с использованием (1).
2) В каждой позиции получить Gx и Gy, применяя оба ядра из (2).
3) Рассчитать длину градиента согласно выражению (3).
4) Ограничить амплитуду значениями 0–255 по правилу (4).

Оператор Робертса

В методе Робертса используются четыре соседних пикселя. Два ядра 2×2 позволяют оценить перепады яркости вдоль диагоналей; их вид указан в (5):

$$G_x=\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}, G_y=\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix} \quad (5)$$

Компоненты отклика можно записать непосредственно через разности яркостей:

$$G_x(x,y)=I(x,y)-I(x+1,y+1), G_y(x,y)=I(x+1,y)-I(x,y+1) \quad (6)$$

Далее две компоненты объединяются по правилу (3), а результат обрезается в соответствии с (4).

Алгоритм 2. Оператор Робертса

1) Подготовить полутоновый растр, воспользовавшись (1).
2) Для текущего пикселя найти обе диагональные разности по (6).
3) Объединить разности в амплитуду градиента согласно (3).
4) Применить ограничение (4) и сохранить вычисленную яркость.

Оператор Лапласа

Лапласиан складывает вторые производные яркости по двум осям. Его непрерывная запись имеет вид (7):

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \qquad (7)$$

Для растра производные заменяются конечными разностями. В работе принята четырёхсвязная маска 3×3 из выражения (8):

$$L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \qquad (8)$$

Отклик может иметь любой знак. Чтобы представить его как яркость границы, используем модуль и ограничиваем диапазон, как показано в (9):

$$I_{out}(x, y) = min\left(255, \left|(L * I)(x, y)\right|\right) \qquad (9)$$

Алгоритм 3. Оператор Лапласа

1) Преобразовать цветной вход в яркостную матрицу по (1).
2) Для каждой позиции вычислить отклик ядра Лапласа из (8).
3) Взять модуль отклика и записать результат с учётом (9).

2.2.Линейные усредняющие фильтры

Случайные отклонения яркости можно ослабить, заменяя отдельный отсчёт средним по соседним точкам. При равных весах получается прямоугольное усреднение, при весах, зависящих от расстояния, — взвешенное сглаживание.

Прямоугольный усредняющий фильтр (Box Blur)

В прямоугольном фильтре все элементы окна участвуют с одинаковым весом. Для нечётного размера k и радиуса r = (k − 1)/2 используется среднее, заданное формулой (10):

$$I_{out}(x,y)=\frac{1}{k^2}\sum_{i=-r}^{r}\sum_{j=-r}^{r} I(x+i,y+j) \qquad (10)$$

Алгоритм 4. Прямоугольное усреднение (Box Blur)

1) Выбрать нечётное k и определить радиус r = (k − 1)/2.
2) Обойти окно каждого пикселя и сложить его k^2 значений яркости.
3) Разделить накопленное значение на площадь k^2, как в (10), и сохранить его.

Фильтр Гаусса (Gaussian Blur)

У фильтра Гаусса вклад соседнего пикселя уменьшается с удалением от центра. Начальные значения весов вычисляются по двумерной функции (11):

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{\frac{-(x^2+y^2)}{2\sigma^2}} \qquad (11)$$

Параметр σ задаёт характерную ширину распределения и влияет на степень размытия.

Из функции строится дискретное ядро G размером k×k. Его элементы делятся на общую сумму весов по правилу (12):

$$w(i,j)=\frac{G(i,j)}{\sum_{u=-r}^{r}\sum_{v=-r}^{r}G(u,v)} \qquad (12)$$

После нормировки сумма коэффициентов равна единице. Выходная яркость получается взвешенным суммированием соседних отсчётов согласно (13):

$$I_{out}(x,y)=\sum_{i=-r}^{r}\sum_{j=-r}^{r}w(i,j)*I(x+i,y+j) \qquad (13)$$

Алгоритм 5. Фильтр Гаусса

1) Указать нечётный размер окна k и положительное значение σ.
2) Посчитать веса по (11), затем привести их сумму к единице по (12).
3) Последовательно применить подготовленное ядро ко всему растру по (13).

3. Программное описание алгоритма

Программа написана на C++17. Хранение пикселей и операции чтения и записи PNG выполняет Image. В классе сохранён заданный интерфейс; обработка вынесена в отдельное пространство Filters. Копирование Image создаёт независимый массив, поддерживается от одного до четырёх каналов.

Filters::applySobel получает неизменяемый исходный Image. После вычисления полутоновой яркости функция накапливает отклики двух масок 3×3. Из них рассчитывается модуль градиента, затем значение ограничивается 255. Это программное выполнение алгоритма 1 и формул (1)–(4). Листинг помещён в приложении А.

В Filters::applyRoberts для каждой позиции считываются четыре яркости соседнего блока 2×2. Разности по диагоналям дают компоненты из (6); их длина становится яркостью выходного пикселя. Функция соответствует алгоритму 2, а её код приведён в приложении А.

Filters::applyLaplacian применяет к полутоновому растру маску (8). Отрицательные и положительные отклики переводятся в неотрицательную яркость взятием модуля, после чего выполняется ограничение по (9). Алгоритм 3 реализован в листинге приложения А.

В Filters::applyBoxBlur параметр kernelSize определяет сторону окна. Функция складывает значения его пикселей и делит сумму на число отсчётов. Для одноканального входа обрабатывается яркость, для RGB — каждый цветовой канал отдельно. Выражение (10) и алгоритм 4 реализованы в приложении Б.

Filters::applyGaussianBlur использует параметры kernelSize и sigma. Сначала создаётся и нормируется массив весов по (11)–(12), затем рассчитывается взвешенная сумма (13). Размер окна проверяется на положительность и нечётность. Код алгоритма 5 находится в приложении Б.

4. Результат работы программы

Тестовый растр 256×256 строится без внешних фотографий: на светлом фоне размещены треугольник, ромб, эллипс и крест. Их яркости различаются. Чистый эталон записывается в test_input/clean.png. Добавление шума N(0, 25^2) с seed=42 формирует test_input/input.png. Эта новая композиция используется во всех сравнениях.

Наклонные стороны треугольника и ромба позволяют сравнить отклик операторов на диагональных границах;

Внешняя дуга эллипса и прямые стороны креста показывают, как методы передают криволинейные и осевые контуры;

Свободный фон и внутренние области фигур служат для наблюдения за шумом, а углы и узкие части креста — за потерей деталей после сглаживания.

Наблюдаемые различия объясняются устройством рассмотренных масок и способом преобразования их отклика:

В граничных позициях используется яркость ближайшего допустимого пикселя. После обработки значения приводятся к диапазону 0–255. Равномерное усреднение отбрасывает дробную часть при делении, а гауссово округляет вычисленную сумму до целого.

На чистом эталоне контуры всех четырёх фигур хорошо различимы; результаты сохранены отдельно с префиксом clean_. У Собела заметны как прямые, так и наклонные участки границ. Поперечное усреднение не устраняет шум полностью: на зашумлённом входе фон также даёт отклик, а сильные границы достигают 255.

Робертс использует самое маленькое окно из рассмотренных методов. Получаемые линии сравнительно тонкие, но при добавлении шума появляются отдельные светлые точки на фоне и внутри фигур. Это следствие вычисления разностей без предварительного усреднения.

Для лапласиана небольшие случайные перепады яркости также дают заметный результат. Поэтому шумный фон содержит много ложных границ. На переходах между фигурой и фоном возникают отклики с обеих сторон; после взятия модуля различие их знаков исчезает.

Окно 5×5 подавляет колебания на фоне сильнее окна 3×3, но сглаживает вершину треугольника, углы ромба и края креста. При гауссовом взвешивании центральные отсчёты влияют сильнее удалённых. Это позволяет иначе распределить сглаживание между внутренними областями и контурами.

Результаты измерений относительно чистого оригинала (MSE; PSNR, дБ; MSE на однородном фоне): шум — 614.55; 20.25; 620.89; окно 3×3 — 119.30; 27.36; 74.80; окно 5×5 — 113.95; 27.56; 34.88; Гаусс 5×5, σ=1,5 — 96.64; 28.28; 38.21.

Для сравнения приняты $MSE = \Sigma(I - I_0)^2/(WH)$ и $PSNR = 10\ \lg(255^2/MSE)$. Отдельно измеряется ошибка фона в области $5 \le x < 25$, $110 \le y < 230$. Чем меньше MSE и выше PSNR, тем ближе результат к эталону. При оценке всего растра учитываются также потери на границах фигур, поэтому сильнейшее усреднение фона не всегда оказывается лучшим по общей ошибке.

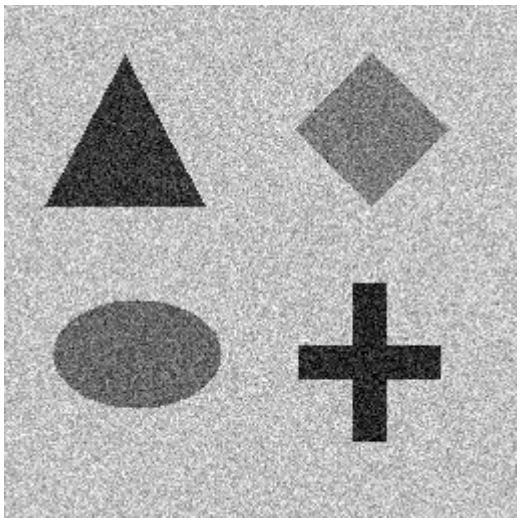


Рис. 1. Программно сгенерированное изображение с гауссовым шумом, σ = 25

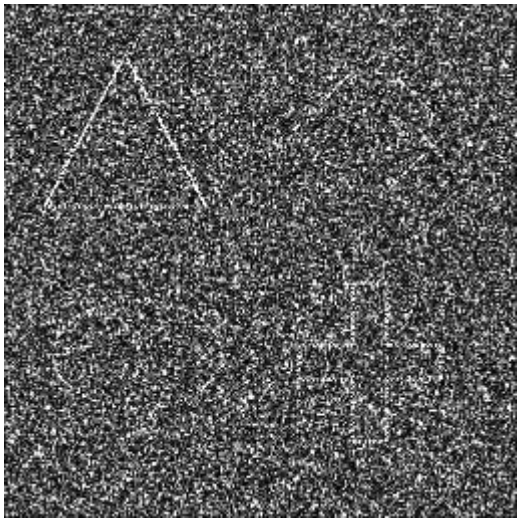


Рис. 2. Результат действия оператора Лапласа

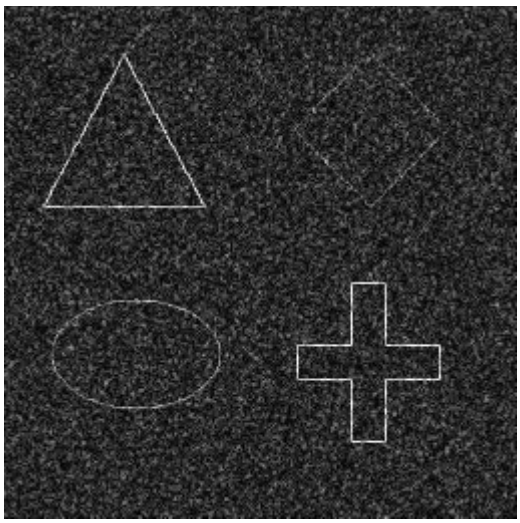


Рис. 3. Результат действия оператора Робертса

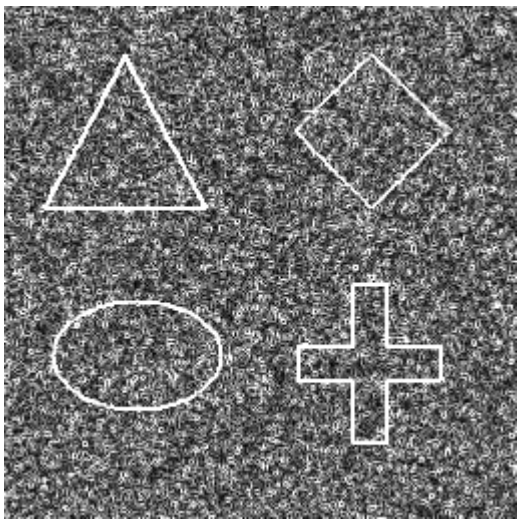


Рис. 4. Результат действия оператора Собела

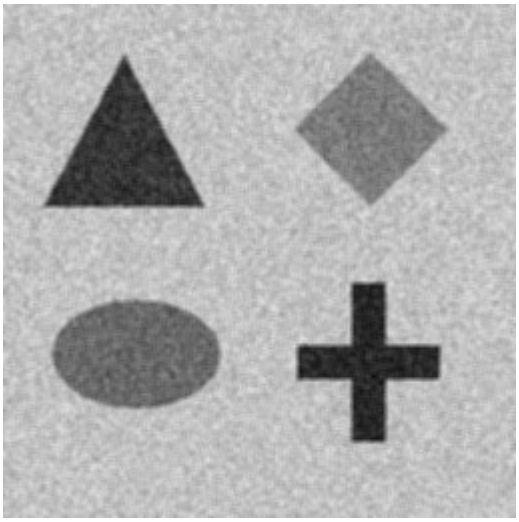


Рис. 5. Прямоугольный усредняющий фильтр 3×3

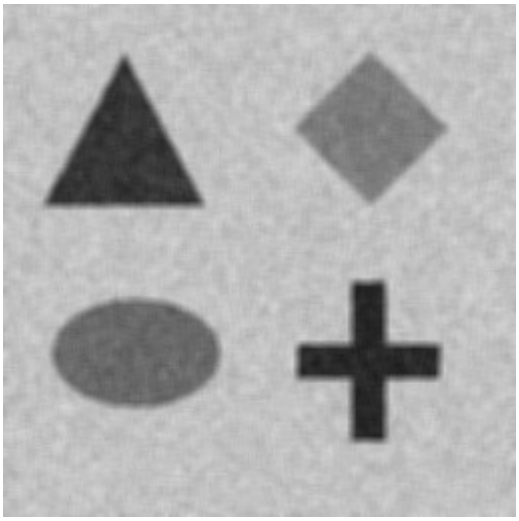


Рис. 6. Прямоугольный усредняющий фильтр 5×5

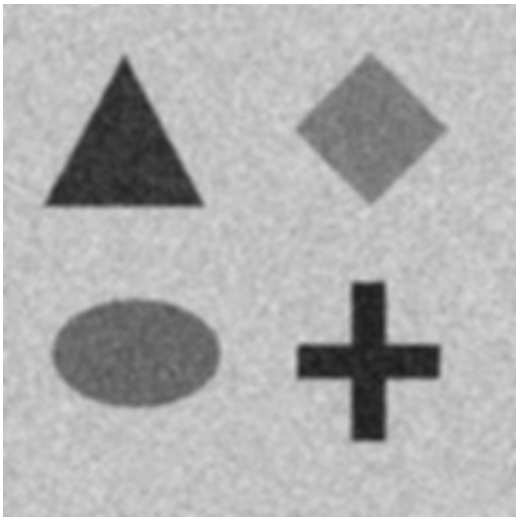


Рис. 7. Усредняющий фильтр Гаусса 5×5, σ = 1,5

5. Заключение

В работе выполнены варианты 1 и 2: получены контуры трёмя дифференциальными операторами и проведено сглаживание гауссова шума двумя видами усреднения. Для эксперимента программно построена собственная композиция геометрических фигур. Обработка выполнена через исходный интерфейс Image без добавления новых методов.

Сравнение полученных изображений позволяет сделать следующие выводы:

Маски Собела сочетают оценку производной и небольшое сглаживание. Они выделяют границы разной ориентации, однако шум также усиливается, а сильные отклики ограничиваются верхним уровнем яркости.

Диагональные разности Робертса дают компактный контур при малом числе операций. Вместе с тем маленькое окно делает результат чувствительным к случайным изменениям яркости соседних пикселей.

Лапласиан реагирует на резкие изменения и может отмечать обе стороны перехода. Без предварительного удаления шума его отклик содержит много посторонних деталей.

Усреднение уменьшило шум на фоне и внутри фигур, но немного размыло их края. На новой сцене фильтр Гаусса показал лучшую общую MSE: 96,64 против 614,55 у шумного входа. Окно 5×5 дало наименьшую ошибку отдельно на ровном фоне. Эти результаты подтверждают, что оценка подавления шума должна учитывать и сохранность контуров.

Приложение А

Реализация функций applySobel, applyRoberts, applyLaplacian:

```
Image applySobel(const Image& src) {
    Image gray = toGrayscale(src);
    int w = gray.width();
    int h = gray.height();
    Image dst(w, h, 1);

    const int Gx[3][3] = {
       {-1, 0, 1},
       {-2, 0, 2},
       {-1, 0, 1}
    };

    const int Gy[3][3] = {
       {-1, -2, -1},
       { 0,  0,  0},
       { 1,  2,  1}
    };

    for (int y = 0; y < h; ++y) {
       for (int x = 0; x < w; ++x) {
          double sumX = 0.0;
          double sumY = 0.0;

          for (int ky = -1; ky <= 1; ++ky) {
             for (int kx = -1; kx <= 1; ++kx) {
                int pixel = getPixelChannel(gray, x + kx, y + ky, 0);
                sumX += pixel * Gx[ky + 1][kx + 1];
                sumY += pixel * Gy[ky + 1][kx + 1];
             }
          }

              int mag = clampByte(static_cast<int>(std::sqrt(sumX * sumX + sumY *
sumY)));
          dst.setColor(x, y, mag, mag, mag);
       }
    }
    return dst;
  }

  Image applyRoberts(const Image& src) {
```

```
    Image gray = toGrayscale(src);
    int w = gray.width();
    int h = gray.height();
    Image dst(w, h, 1);

    for (int y = 0; y < h; ++y) {
        for (int x = 0; x < w; ++x) {
            int p00 = getPixelChannel(gray, x,     y,     0);
            int p11 = getPixelChannel(gray, x + 1, y + 1, 0);
            int p01 = getPixelChannel(gray, x + 1, y,     0);
            int p10 = getPixelChannel(gray, x,     y + 1, 0);

            double gx = p00 - p11;
            double gy = p01 - p10;

            int mag = clampByte(static_cast<int>(std::sqrt(gx * gx + gy * gy)));
            dst.setColor(x, y, mag, mag, mag);
        }
    }
    return dst;
}

Image applyLaplacian(const Image& src) {
    Image gray = toGrayscale(src);
    int w = gray.width();
    int h = gray.height();
    Image dst(w, h, 1);

    const int kernel[3][3] = {
        { 0,  1,  0 },
        { 1, -4,  1 },
        { 0,  1,  0 }
    };

    for (int y = 0; y < h; ++y) {
        for (int x = 0; x < w; ++x) {
            int sum = 0;
            for (int ky = -1; ky <= 1; ++ky) {
                for (int kx = -1; kx <= 1; ++kx) {
                    int pixel = getPixelChannel(gray, x + kx, y + ky, 0);
                    sum += pixel * kernel[ky + 1][kx + 1];
                }
            }
```

```
            int val = clampByte(std::abs(sum));
            dst.setColor(x, y, val, val, val);
        }
    }
    return dst;
}
```

Приложение Б

Реализация функций applyBoxBlur, applyGaussianBlur:

```
Image applyBoxBlur(const Image& src, int kernelSize) {
    if (kernelSize <= 0 || kernelSize % 2 == 0 || kernelSize > 255) throw
std::invalid_argument("Odd kernel size 1..255 required");
    int w = src.width();
    int h = src.height();
    int channels = src.nChannels();
    Image dst(w, h, channels);
    int radius = kernelSize / 2;
    int count = kernelSize * kernelSize;

    for (int y = 0; y < h; ++y) {
      for (int x = 0; x < w; ++x) {
        int sumR = 0, sumG = 0, sumB = 0;

        for (int ky = -radius; ky <= radius; ++ky) {
          for (int kx = -radius; kx <= radius; ++kx) {
            int r = 0, g = 0, b = 0;
            src.getColor(clampCoord(x + kx, w), clampCoord(y + ky, h), r, g, b);
            sumR += r;
            sumG += g;
            sumB += b;
          }
        }

        dst.setColor(x, y, sumR / count, sumG / count, sumB / count);
      }
    }
    return dst;
  }

  Image applyGaussianBlur(const Image& src, int kernelSize, double sigma) {
    if (kernelSize <= 0 || kernelSize % 2 == 0 || kernelSize > 255) throw
std::invalid_argument("Odd kernel size 1..255 required");
    if (!std::isfinite(sigma) || sigma < 1e-6 || sigma > 1e6) throw
std::invalid_argument("sigma must be finite, 1e-6..1e6");
    int w = src.width();
    int h = src.height();
    int channels = src.nChannels();
    Image dst(w, h, channels);
    int radius = kernelSize / 2;
```

```
    std::vector<std::vector<double>> kernel(kernelSize,
std::vector<double>(kernelSize));
    double sumWeights = 0.0;
    double s = 2.0 * sigma * sigma;

    for (int y = -radius; y <= radius; ++y) {
      for (int x = -radius; x <= radius; ++x) {
        double r = x * x + y * y;
        double val = std::exp(-r / s) / (3.14159265358979323846 * s);
        kernel[y + radius][x + radius] = val;
        sumWeights += val;
      }
    }

    for (int i = 0; i < kernelSize; ++i) {
      for (int j = 0; j < kernelSize; ++j) {
        kernel[i][j] /= sumWeights;
      }
    }

    for (int y = 0; y < h; ++y) {
      for (int x = 0; x < w; ++x) {
        double accR = 0.0, accG = 0.0, accB = 0.0;

        for (int ky = -radius; ky <= radius; ++ky) {
          for (int kx = -radius; kx <= radius; ++kx) {
            int r = 0, g = 0, b = 0;
            src.getColor(clampCoord(x + kx, w), clampCoord(y + ky, h), r, g, b);
            double weight = kernel[ky + radius][kx + radius];
            accR += r * weight;
            accG += g * weight;
            accB += b * weight;
          }
        }

        dst.setColor(x, y, clampByte(static_cast<int>(accR + 0.5)),
                     clampByte(static_cast<int>(accG + 0.5)),
                     clampByte(static_cast<int>(accB + 0.5)));
      }
    }
    return dst;
  }
```